

Problem 12 – Smart Car Rental System 🚗

Difficulty: ★★☆☆ Beginner+

1. Overview

A car rental company wants to replace its paper booking system with a digital rental application.

Customers should be able to:

- View available cars.
- Rent a car.
- Choose the number of rental days.
- Choose a rental start date.
- Add optional GPS navigation.
- Receive a rental summary.
- Return a rented car.

The company should also be able to track rentals and generate a rental management report.

🔒 **Streamlit only:** Do not use `input()`, `print()`, or `while True`.

2. Available Cars

Create a dictionary containing the available cars and their **daily rental prices**.

Car	Price Per Day (EGP)
Toyota Corolla	850
Hyundai Elantra	950
Kia Sportage	1400
Nissan Sunny	800
BMW X3	2500

Create a separate way to track whether each car is:

- **Available**
- **Rented**

Initially, all cars must be available.

3. Application Layout

Your Streamlit application must contain:

Sidebar

Display:

- Rental rules
- Daily rental prices
- A short rental-company information section

The complete car availability list belongs in the **Available Cars tab**, not the sidebar.

Four Tabs

1. **Rent a Car**
2. **Available Cars**
3. **Return a Car**
4. **Rental Summary**

Use `st.tabs()`.

4. Rent a Car Tab

Create an `st.form()` containing the rental controls.

Customer Name Use `st.text_input()`.

Car Use `st.selectbox()` to allow the customer to choose a car.

Rental Days Use `st.number_input()`.

The number of rental days must be greater than zero.

Rental Start Date Use `st.date_input()`.

GPS Navigation Use `st.selectbox()` with:

- Yes
- No

GPS costs **150 EGP per rental**.

Confirm Rental Use `st.form_submit_button()`.

5. Rental Validation

When the customer confirms the rental:

- Customer name must not be empty.
- The selected car must exist.
- The selected car must be available.
- Rental days must be greater than zero.
- A rental date must be selected.

If the information is invalid, do not create the rental.

Use `st.error()` to display the appropriate error message.

If the selected car is already rented, display a warning using `st.warning()`.

6. Calculate Rental Cost

Calculate the basic rental cost using:

Rental Cost = Daily Price × Number of Rental Days

If GPS is selected:

Final Cost = Rental Cost + 150 EGP

If GPS is not selected:

Final Cost = Rental Cost

Example

Hyundai Elantra:

- Daily price = 950 EGP
- Rental days = 4
- GPS = 150 EGP

Rental cost:

950 × 4 = 3800 EGP

Final cost:

3800 + 150 = 3950 EGP

7. Confirm the Rental

After a valid rental:

1. Calculate the rental cost.
2. Add the GPS cost if selected.
3. Mark the car as **Rented**.
4. Store the rental information.
5. Display a confirmation message.
6. Make the rental available in the **Rental Summary** tab.

Use:

- `st.spinner()` to simulate processing the booking.
- `st.success()` after the rental is successfully confirmed.

Example:

 Confirming your rental...

Then:

 Rental confirmed successfully!

8. Available Cars Tab

Display **all cars** and their:

- Car name
- Daily price
- Rental status

The status must clearly show:

- Available
- Rented

Example:

Car	Price/Day	Status
Toyota Corolla	850 EGP	Available
Hyundai Elantra	950 EGP	Rented
Kia Sportage	1400 EGP	Available

The information should update after a car is rented or returned.

9. Return a Car Tab

Create a section that allows an employee to return a rented car.

The employee should provide:

Customer Name

Use `st.text_input()`.

Car

Use `st.selectbox()` to choose the car being returned.

Return Car

Use `st.button()`.

When the return button is pressed:

- Validate the customer information.
- Check whether the selected car is currently rented.
- If it is rented, change its status to **Available**.
- Display a confirmation message.

Use:

- `st.success()` for a successful return.
 - `st.warning()` if the car is already available.
 - `st.error()` if the customer information is invalid.
-

10. Rental Summary Tab

Display the **most recently confirmed rental**.

If no rental has been made yet, display an information message using `st.info()`.

Use `st.columns(2)`.

Left Column – Rental Information

Display:

- Customer name
- Car
- Rental days
- Rental start date

Right Column – Cost Information

Display:

- Price per day
 - Basic rental cost
 - GPS status
 - GPS cost
 - Final total
 - Rental status
-

11. Rental Summary

Use `st.code()` to display a clear formatted rental summary.

Example:

```
=====
```

```
Rental Summary
```

```
=====
```

```
Customer : Ahmed
```

```
Car : Hyundai Elantra
```

```
Rental Days : 4
```

```
Rental Date : 06/07/2026
```

```
GPS : Added
```

```
Final Cost : 3950 EGP
```

```
Status : Confirmed
```

```
=====
```

12. Required Text Functions

Your application must use **all six**:

Function	Purpose
st.title()	Application title
st.header()	Major sections
st.subheader()	Smaller sections
st.write()	General information
st.code()	Rental summary
st.divider()	Separate sections

13. Required Input Widgets

Your application must use:

Widget	Purpose
st.text_input()	Customer name
st.selectbox()	Car selection
st.number_input()	Rental days
st.date_input()	Rental start date
st.selectbox()	GPS Yes/No
st.form_submit_button()	Confirm rental
st.button()	Return car

All rental controls must be inside an `st.form()`.

The **Return Car** action should use a separate `st.button()` in the Return tab.

14. Required Status Elements

You must use **all five**:

Function	Purpose
st.info()	Rental instructions / no rental yet
st.success()	Rental or return confirmed
st.warning()	Car already available/rented
st.error()	Invalid information
st.spinner()	Simulate rental processing

15. Store Rentals

Store every successfully created rental in a **list of dictionaries**.

Each rental should contain information such as:

- Customer name
- Car
- Rental days
- Rental date
- Daily price
- GPS selection
- GPS cost
- Final cost

Because Streamlit reruns the application when users interact with it, store the rentals and car statuses using `st.session_state`.

The information must remain available when the user:

- Changes tabs.
 - Places another rental.
 - Returns a car.
-

16. Required Functions

Create at least these functions:

display_cars()

Displays the cars, prices, and availability status.

calculate_rental_cost()

Calculates the rental cost and optional GPS cost.

rent_car()

Handles the rental process and updates the car's status.

return_car()

Handles returning a car and changing its status back to Available.

display_rental_summary()

Displays the latest rental information and final cost.

You may create additional functions if needed.

17. Challenge Mission ★★

Rental Management Dashboard

Create a **Rental Management Dashboard** using the stored rental information.

Display:

Total Rentals

The total number of successful rentals.

Total Income

The total income from all confirmed rentals.

Most Rented Car

The car that has been rented the greatest number of times.

Available Cars

The number or list of cars currently available.

Rented Cars

The number or list of cars currently rented.

Average Rental Duration

Calculate the average number of rental days.

Customer With the Highest Rental Bill

Find the customer associated with the highest rental cost.

18. Required Streamlit Topics

Displaying Text

- `st.title()`
- `st.header()`
- `st.subheader()`
- `st.write()`
- `st.code()`
- `st.divider()`

Input

- `st.text_input()`
- `st.selectbox()`
- `st.number_input()`
- `st.date_input()`
- `st.form()`
- `st.form_submit_button()`
- `st.button()`

Status

- `st.info()`
- `st.success()`
- `st.warning()`
- `st.error()`
- `st.spinner()`

Layout

- `st.sidebar`
- `st.tabs()`
- `st.columns()`

State

- `st.session_state`
-

19. Python Topics

This project practices:

- Variables
 - Strings
 - Lists
 - Dictionaries
 - Integers and floats
 - Arithmetic operators
 - Comparison operators
 - Assignment operators
 - Membership operators
 - if / elif / else
 - for loops
 - User-defined functions
 - Dictionary methods
 - List operations
 - datetime / date handling
-

20. Final Checklist

Before submitting, make sure you have:

Data

- Car price dictionary
- Car availability status
- Rental records list
- Data stored using `st.session_state`

Sidebar

- Rental rules
- Daily prices
- Company information

Tabs

- Rent a Car
- Available Cars
- Return a Car
- Rental Summary

Rental Form

- Customer name
- Car selection
- Rental days
- Rental date
- GPS option
- Confirm Rental

Rental System

- Input validation
- Rental cost calculation
- Optional GPS
- Car becomes Rented
- Car can be returned
- Returned car becomes Available
- Rental summary

Required Streamlit Components

- All required text functions
- All required input widgets
- All five status elements
- Sidebar
- Four tabs
- Form
- Two-column summary
- Session state

Challenge

- Total rentals
 - Total income
 - Most rented car
 - Available cars
 - Rented cars
 - Average rental duration
 - Highest rental bill
-

21. Important Restrictions

This project must be built as a **Streamlit application**.

Do **not** use:

- `input()`
- `print()`
- `while True`
- Console menus

Streamlit handles user interaction and application reruns automatically.

22. Learning Objectives

After completing this project, you should be able to:

- Manage real-world inventory using dictionaries.
- Track changing availability using Streamlit state.
- Build a complete rental workflow.
- Validate user information.
- Work with dates.
- Calculate rental costs and optional services.
- Update data when cars are rented or returned.
- Store multiple rental records.
- Build a multi-tab Streamlit application.
- Use forms, sidebars, columns, status messages, and session state.
- Generate a basic rental management dashboard.